



Architecture Technique et Sécurité - ChadiBot (V4 Production Massive)

1. Vue d'ensemble de l'Infrastructure et Identité de Production

L'architecture du projet ChadiBot a été intégralement repensée pour supporter une échelle industrielle : **1,1 million de fichiers, 121 000 dossiers, et 2,25 To de données** répartis sur l'arborescence /services. Elle garantit l'intégrité des données, la performance des traitements, et un respect strict des droits d'accès via une approche "Zero Trust" pré-calculée.

- **faf-fls (Serveur de fichiers)** : Héberge le lecteur réseau global (racine D:\Services). Il exécute l'agent de surveillance documentaire asynchrone (Watcher).
- **srv-llm01 (Serveur IA)** : Machine Virtuelle dédiée hébergeant l'interface Streamlit, le routeur Flask (Webhook), la nouvelle base relationnelle chadibot_v4.db, l'index vectoriel FAISS, et les modèles de langage locaux (Ollama / Llama 3.2).
- **Identité de Service et Privilèges** : Les scripts opèrent exclusivement sous le compte de service géré **gMSA (FAF-FR\gmsa_chadibot\$)**, assurant une rotation automatique des mots de passe. L'exécution se fait en **Session 0** via le Planificateur de tâches Windows, avec les droits NTFS de lecture/exécution (RX) sur les sources de données cibles.

2. Déploiement Client et Protocole d'Exécution (GPO)

Pour permettre l'ouverture directe des fichiers depuis l'interface Web vers l'explorateur Windows du poste client (sans passer par un téléchargement HTTP classique), un protocole URI personnalisé (doc-ia:) est déployé.

- **Objet GPO** : l-Déploiement_ChadiBot (ID: {590DB7E6-7F68-4766-91D7-07C186C2D686}).
- **Périmètre de Déploiement** : La GPO est déployée pour les ordinateurs du domaine (faf.lan/FR/Computers/SI et OUs cibles), avec un filtrage de sécurité appliqué aux "Utilisateurs authentifiés".
- **Technique** : Déploiement de clés de registre HKLM\SOFTWARE\Classes\doc-ia définissant le URL Protocol.
- **Exécution Sécurisée** : Le protocole appelle powershell.exe -WindowStyle Hidden -ExecutionPolicy Bypass -File "\\faf-fls\services\INFORMATIQUE\lanceur_ia.ps1" "%1", utilisant un jeton secret pour autoriser l'action (OPEN, EDIT, ou RUN).

3. Ingestion Initiale et Base Hybride (Le Cerveau SQL/Vectoriel)

L'ingestion des 2,25 To est structurée autour d'une base relationnelle SQLite optimisée (mode

WAL - Write-Ahead Logging) séparant les entités "Dossiers" et "Fichiers".

- **Le Routeur Hybride** : Le système sépare strictement les données.
 - **Fichiers VIP (IA)** : Les formats documentaires (.pdf, .docx, .txt, .py, .ps1...) voient leur contenu envoyé à l'IA pour générer un vecteur sémantique FAISS (est_indexable_faiss = 1).
 - **Fichiers Binaires (Catalogue)** : Les exécutables (.exe, .msi) et images sont catalogués dans SQLite uniquement avec leurs métadonnées (est_indexable_faiss = 0), évitant la corruption du modèle mathématique.
- **Lecture NTFS Native (Le Cache ACL)** : L'extraction des droits abandonne PowerShell au profit de l'API C++ native de Windows (pywin32 / win32security). Le script lit la Master File Table (MFT) pour extraire les Descripteurs de Sécurité (DACL) à la milliseconde.
- **Filtre Logique et Résolution SID** : Seules les règles accordant explicitement le droit FILE_READ_DATA sont conservées. Les SIDs sont résolus en noms de groupes via LookupAccountSid et stockés dans un cache RAM (CACHE_SID_NOM) pour protéger le Contrôleur de Domaine.
- **Persistance Relationnelle** : Les droits sont stockés sous forme de liste JSON (acl_sids) dans la table dossiers. La table fichiers est liée par dossier_id, évitant la redondance massive de données.

4. Surveillance Asynchrone et Architecture Delta

Le maintien à jour du lac de données est géré par une architecture événementielle "Delta".

- **Watcher PowerShell Musclé (Watcher_IA.ps1)** : Le script s'abonne aux événements de l'OS via System.IO.FileSystemWatcher ciblant D:\Services. Pour supporter la charge de la racine complète, le buffer interne (InternalBufferSize) est poussé à son maximum (**64 Ko**). Les événements sont traités de manière asynchrone (Session 0).
- **Webhook Flask (api_webhook.py)** : Intercepte les requêtes JSON du Watcher.
 - **Anti-Fichiers Fantômes** : Si un fichier est verrouillé ou transitoire (ex: enregistrement Word), la fonction obtenir_groupes_dossier remonte récursivement l'arborescence (os.path.dirname) jusqu'à trouver un dossier parent valide pour hériter de ses ACLs.
 - **Action Chirurgicale** : Ajout, mise à jour (UPDATE fichiers SET...) ou suppression en base.
- **FAISS Delta** : Si le fichier est "VIP", l'index vectoriel IndexIDMap est mis à jour instantanément en RAM, sans nécessiter de recalcul global de la base.

5. Portail d'Entrée et SSO Silencieux

L'authentification a été refondue pour éliminer les latences réseau lors des recherches, adoptant une stratégie de "Passeport VIP".

- **Sas IIS Isolé** : Le serveur web IIS (Reverse Proxy) valide silencieusement l'identité via NTLM/Kerberos (DefaultAppPool) et génère un jeton de session (.tok) lu par Streamlit.
- **Extraction Active Directory Optimisée** : À la connexion, app.py utilise win32net.NetUserGetGroups et NetUserGetLocalGroups pour extraire instantanément la

totalité des groupes AD de l'utilisateur (globaux et locaux). Ce trousseau de clés est stocké en session Streamlit (st.session_state.groupe_ad).

6. Moteur de Recherche Hybride et Arbitrage RRF

La compréhension cognitive des requêtes utilisateur est confiée à un LLM local.

- **Extraction d'Intention (Ollama - Llama 3.2) :** Transforme le langage naturel en requêtes JSON strictes. L'IA injecte automatiquement les extensions pertinentes (ex: ajout de .exe et .msi si le mot install est détecté). Un "Failsafe" garantit la conservation des entités nommées d'origine en cas d'hallucination du modèle.
- **Recherche Duale :**
 - **Lexicale (SQLite) :** Recherche agnostique hyper-large (LIKE) sur les noms de fichiers ET les chemins de dossiers.
 - **Sémantique (FAISS) :** Recherche par similarité vectorielle (MiniLM-L12-v2), avec un partitionnement optimisé (index_faiss.nprobe = 10).
- **Reciprocal Rank Fusion (RRF) :** L'algorithme RRF fusionne les résultats des deux moteurs et applique des multiplicateurs de pertinence stricts :
 - Pénalité massive (0.01x) si l'extension exigée ne correspond pas.
 - Bonus exponentiel (2^n) pour la correspondance exacte des mots dans le titre.
 - "God Mode" (500x) si le nom du fichier est un clone parfait de la requête.

7. Le "Bouclier NTFS" : Sécurité Zero Trust Hybride

L'innovation majeure de la V4 réside dans son mécanisme de vérification des droits (Post-Filtering) hybride, conçu pour allier la rapidité du cache SQLite à la fiabilité de l'audit système.

Lorsqu'un document est présélectionné par le moteur de recherche, la fonction `a_le_droit_de_lire` s'active :

1. **Tentative Temps Réel (L'Épée) :** Appel direct à `win32security.GetFileSecurity` sur le chemin réseau (\\f-fs\Services\...). Le système croise les ACEs (Access Control Entries) avec les groupes AD en session de l'utilisateur. Si un `ACCESS_DENIED_ACE_TYPE` (Refus explicite) est rencontré, le fichier est bloqué immédiatement.
2. **Fallback SQLite (Le Bouclier) :** Si l'accès réseau échoue (ex: erreur système 5 Accès Refusé), le système interroge le cache NTFS pré-calculé dans SQLite (`SELECT acl_sids FROM dossiers`).
3. **Sanctuaire Anti-Fuite (Data Loss Prevention) :** Pour les répertoires critiques (RH, Direction, Compta, Service Médical...), le Fallback SQLite est **strictement interdit**. Si l'audit réseau en direct échoue sur ces zones, le fichier est écarté par défaut.

Si un document ne passe pas ce bouclier, il est purgé des résultats finaux. L'utilisateur ne verra jamais l'existence d'un fichier qu'il n'a pas le droit d'ouvrir.